Trabalho Prático 2 – Organização de Computadores 1

Bernardo Borges Machado
2023111115

Leonardo César Cota de Castro
2023087923

Luis Felipe Pimenta Marcos
2023087907

Departamento de Ciência da Computação – Universidade Federal de Minas Gerais (UFMG) Belo Horizonte – MG – Brasil

https://colab.research.google.com/drive/1dJBcQ5op1nxrnIkIMUWq1FEzmm4qZtAz?usp=drive_link

---

1. Introdução

Este relatório tem como objetivo abordar o desenvolvimento desse trabalho em Verilog, focando em quatro instruções principais: as duas primeiras (add e rem) são do tipo-R, a terceira (ori) é do tipo-I e a última (beq) é uma do tipo-B.

Durante a execução do trabalho, alteramos apenas as unidades de Decodificação e Execução, pois as demais já estavam corretamente implementadas.

Na instrução add, alteramos o modulo denominado alucontrol, pertencente à unidade de execução.

Na instrução rem, alteramos os módulos alucontrol e ALU, ambos pertencentes à uinidade de execução.

Na instrução ori, alteramos apenas o módulo alucontrol, pertencente à unidade de execução.

Finalmente, na instrução beq, alteramos apenas o módulo ControlUnit, pertencente à unidade de Decodificação.

2. Desenvolvimento

2. 1- O que foi adicionado na primeira implementação (add)

```
0: begin
   if (funct7 == 7'b0000001) alucontrol <= 4'd10; // MUL
   else if (funct7 == 7'b0100000) alucontrol <= 4'd6; // SUB
   else alucontrol <= 4'd2; // ADD
end
```

Explicação breve do código

Bloco if- else acionado quando o aluop é igual a 2 e o funct3 é igual a 0. Ele verifica o funct7 para definir se a operação lida é de multiplicação, subtração ou soma, sendo essa última o caso geral após as duas primeiras verificações.

Na instrução add, a única unidade que foi alterada foi a unidade de execução, no módulo alucontrol

Fluxo de raciocínio

Ao analisar o módulo denominado alucontrol, percebemos que, ao verificar o funct3 como zero, o código interpretava como sendo uma instrução do tipo R (registrador – registrador) e então passava a verificar o funct7 para decidir entre as operações de subtração e multiplicação, mas não a de adição.

Nesse caso queríamos adicionar essa instrução, então mudamos o bloco de código da sessão que fazia a verificação por meio de um operador ternário para um bloco if-else. Esse bloco verificava se o funct7 era o da multiplicação, se não fosse verificava se era o da subtração, e caso não fosse os dois ele entrava em um caso padão, definido como:

- “sempre que nenhuma das condições anteriores — sejam elas os diferentes valores de aluop ou dos cases de funct3/funct7 — for satisfeita, o controle da ALU cairá no valor 4'd2, que corresponde à operação de soma no módulo ALU”.

A unidade de decode já estava preparada para instruções do tipo R e o módulo da ALU também da realizava a operação ADD. Assim, precisamos apenas modificar o alucontrol.

Fluxo simplificado

1. CPU lê a instrução ADD (opcode R, funct3 = 000).
2. No módulo de controle principal, ela gera aluop = 2 (operações do tipo R).
3. Entra em case (aluop) no módulo alucontrol.
4. No case funct3 = 000, verifica funct7 e define alucontrol <= adição no fallback.
5. A ALU recebe alucontrol = 0000 e executa a operação A + B.

2.2- O que foi adicionado na segunda implementação (rem)

```
6: begin
   if (funct7 == 7'b0000001) alucontrol <= 4'd11; // REM
   else alucontrol <= 4'd1; // OR
end
```

Explicação breve do código

- Decodificação em alucontrol:

Antes, no caso funct3 == 6, o código só fazia a operação OR.

Agora, se além de funct3 == 6 o campo funct7 for 0000001, reconhecemos a instrução REM (funct3 = 110 e funct7 = 0000001) e gera o código de controle 11 em vez de 1.

Se não for funct7 = 0000001, mantém o OR normal.

Implementação no datapath da ALU:

Acrescentando esse case com alucontrol == 11, a ALU passa a calcular A % B (resto da divisão inteira) sempre que o sinal de controle vier igual a 11, como mostrado abaixo.

```
11: aluout <= A % B; // REM
```

Na instrução REM, apenas a unidade de execução foi alterada, mas alteramos ambos os módulos alucontrol e ALU.

Fluxo de raciocínio

Ao analisar o módulo chamado alucontrol, percebemos que, no case que tratava funct3 = 6 era tratada apenas a operação OR, mas faltava a REM. Essa sessão de código foi expandida para um bloco if-else. Agora, é verificado se funct7 == 7'b0000001, interpreta como instrução REM e atribui alucontrol = 4'd11 ou mantém o comportamento antigo e atribui alucontrol = 4'd1.

Em seguida, acrescentamos no case alucontrol (11: aluout <= A % B;). Isso faz com que, sempre que o código de controle seja 11, a ALU execute aluout = A % B—calculando o resto da divisão inteira—ao invés de qualquer outra operação.

Portanto, tanto a geração do sinal de controle alucontrol = 11 quanto a implementação da operação no datapath (aluout <= A % B) foram incluídas, permitindo que a ALU suporte agora a instrução REM do conjunto M do RISC-V.

A unidade de decode já estava preparada para instruções do tipo R, então não precisou ser alterada.

Fluxo simplificado

1. CPU lê a instrução REM (opcode R, funct3 = 110, funct7 = 0000001).
2. No módulo de controle principal, ela gera aluop = 2 (operações do tipo R).
3. Entra em case (aluop) no alucontrol.
4. No subcase funct3 = 6, verifica funct7: Se funct7 == 7'b0000001, faz alucontrol <= 4’d11. Caso contrário, mantém alucontrol <= 4’d1 (instrução OR).
5. A ALU recebe alucontrol = 1011 e executa a operação A % B.

2.3- O que foi adicionado na terceira implementação (ori)

```
6: alucontrol <= 4'd1; // ORI
```

Explicação breve do código

- Decodificação em alucontrol:

No case 6 (alucontrol <= 1;), assim que o decodificador reconhece um ORI (funct3 = 6), ele define alucontrol para 1. Posteriormente, a própria ALU verá esse código 0001 e executará a operação bit-a-bit OR entre o registrador fonte e o imediato.

Na instrução ORI, a única unidade que foi alterada foi a unidade de execução, mais especificamente o bloco de controle de operações da ALU.

Fluxo de raciocínio

Ao analisar o módulo denominado alucontrol, percebemos que, ao receber aluop como três, ele já entrava na sessão que trata das instruções do tipo imediato (I-type). Nesse sub-bloco, cada valor de funct3 mapeava para uma operação diferente da ALU.

Nesse caso, queríamos adicionar a instrução de OR com imediato, cujo funct3 é 6. Para isso, incluímos a linha (6: alucontrol <= 4'd1) dentro do caminho imediato.

Com essa mudança, quando o decodificador principal gera aluop = 3 e o campo funct3 da instrução é igual a 6, o alucontrol passa a atribuir o código interno 4'd1 à saída alucontrol. Em seguida, a própria ALU reconhece esse código 0001 e executa a operação lógica OR entre o valor do registrador fonte e o imediato.

A unidade de decode já estava preparada para instruções do tipo imediato, e a ALU interna já implementava a operação OR, portanto não foi necessário mexer em mais nenhum módulo além do alucontrol.

Fluxo simplificado

1. CPU lê a instrução ORI (opcode I, funct3 = 110).
2. No módulo de controle principal, ela gera aluop = 3 (caminho das imediatas).
3. Entra em case (aluop) no seu alucontrol.
4. No sub-case (funct3), chega em 6: e faz alucontrol <= 4'd1.
5. A ALU recebe alucontrol = 0001 e executa a operação A | imm.

2.4- O que foi adicionado na quarta implementação (beq)

```
7'b1100011: begin
   branch <= 1;
   aluop <= 1;
   ImmGen <= {{19{inst[31]}}, inst[31], inst[7], inst[30:25], inst[11:8], 1'b0};
end
```

Explicação breve do código

(branch <= 1) Sinaliza que estamos em uma instrução de desvio. Na fase de execução, esse "branch" será usado para escolher entre PC+4 ou PC+offset.

(aluop <= 1; Força a ALU a fazer uma subtração (SUB). Como os desvios condicionais em RISC-V comparam dois registradores, a ideia é calcular rs1 – rs2 e, se o resultado for zero (para BEQ) ou negativo (para BLT), ativar o ramo.

Os bits de imediato na instrução B-type estão espalhados em inst[31], inst[7], inst[30:25] e inst[11:8]. O código concatena esses pedaços, insere um 0 e faz sign-extension para 32 bits.

Na instrução de desvio do tipo-B, a única unidade que foi alterada foi a unidade de controle.

Fluxo de raciocínio

Ao analisar o módulo, percebemos que, ao detectar o opcode igual a 7'b1100011, o código passa a tratar a instrução como um branch e então:

Sinaliza o desvio com (branch <= 1;) para que, na fase de decisão do PC, o hardware saiba que deve escolher entre PC + 4 ou PC + offset.

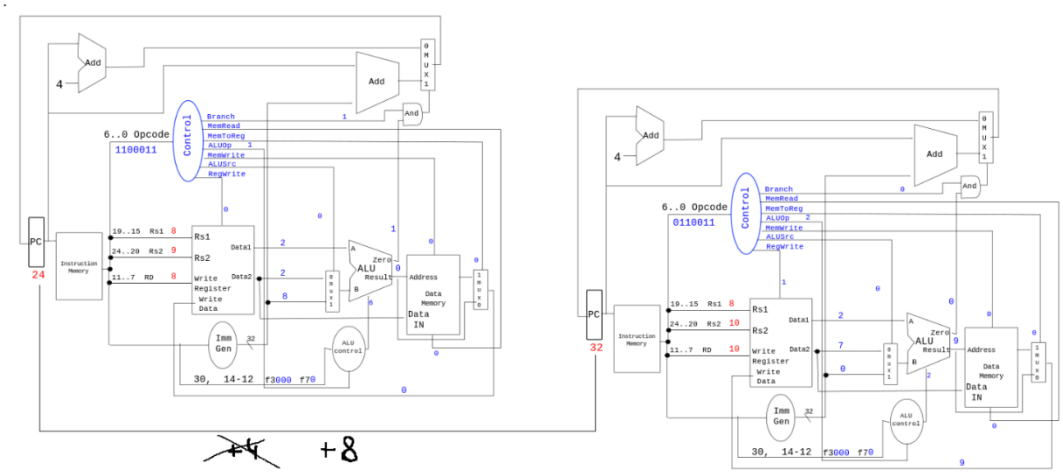
Ajusta (aluop <= 1;) — força a ALU a realizar uma subtração para comparar rs1 e rs2.

Gera o imediato de 13 bits espalhado nos campos da instrução, concatenando e fazendo sign-extension:

A unidade de Decode já sabia lidar com instruções de tipo B (bastava reconhecer o opcode) e a ALU já realiza subtração, portanto não foi necessário mexer nelas — bastou incluir esse bloco if-case no ControlUnit para que todo o suporte a branches condicionais (BEQ, BNE, BLT, etc.) estivesse pronto.

Fluxo simplificado

1. CPU lê a instrução BEQ (opcode B = 1100011).
2. No módulo de controle principal, detecta o opcode e sinaliza branch <= 1.
3. Gera aluop = 1 para que a ALU realize uma subtração entre rs1 e rs2.
4. A unidade de decode extrai o imediato de 13 bits (com concatenação e sign extension).
5. A ALU compara A - B; se o resultado for zero (BEQ), PC recebe PC + offset; senão, PC + 4.



No teste fica claro o salto com o PC saindo de 24 até 32, pulando uma instrução no caminho.

3. Testes realizados

```
addi s0, zero, 2
addi s1, zero, 3
add a0, s0, s1 #5
rem a0, s1, s0 #1
ori a0, s1, 5 #7

addi s1, zero, 2
beq s0, s1, fim
add a0, s0, s1 #4

fim:
add a0, s0, a0 #9
```

1. addi s0, zero, 2
    - Carrega o imediato 2 no registrador s0.
    - Depois desta instrução: s0 ← 0 + 2 = 2
    - addi já está implementado.
2. addi s1, zero, 3
    - Carrega o imediato 3 no registrador s1.
    - Estado: s1 ← 0 + 3 = 3
3. add a0, s0, s1
    - Operação R-type, campo funct3 = 000, funct7 = 0000000, opcode 0110011.
    - O multiplexador de entrada do segundo operando escolhe o segundo registrador (não imediato), o ALU faz soma.
    - Calcula a0 ← s0 + s1 = 2 + 3 = 5.
4. rem a0, s1, s0 # comentário: 1
    - Operação R-type de resto, opcode 0110011, funct3 = 110, funct7 = 0000001.
    - A unidade de divisão calcula quociente e resto de s1 ÷ s0. O sinal de controle escolhe saída resto.
    - Como s1 = 3, s0 = 2, então resto = 3 % 2 = 1.
    - Resultado: a0 ← 1

5. ori a0, s1, 5
    - Operação I-type, opcode 0010011, funct3 = 110.
    - O imediato de 12 bits é zeroestendido e aplicado num bit-wise OR com o valor de s1.
    - s1 = 3 (0b011), imediato = 5 (0b101), logo: a0 ← 0b011 OR 0b101 = 0b111 = 7
6. addi s1, zero, 2
    - Reatribui s1 ← 2 (mesmo mecanismo do passo 1).
    - Estado agora: s0 = 2, s1 = 2, a0 = 7
7. beq s0, s1, fim
    - Operação B-type, opcode 1100011, funct3 = 000.
    - Compara s0 e s1. Se iguais, PC ← PC + 4 + offset.
    - Aqui, s0 = s1 (2 = 2), condição verdadeira. O offset saltará diretamente para o rótulo fim, pulando a próxima instrução.
8. (Pulada) add a0, s0, s1
    - Esta instrução é ignorada porque o branch foi tomado.
9. fim:
    - Rótulo de destino do beq.
10. add a0, s0, a0
    - Mais uma operação R-type de soma: a0 ← s0 + a0 = 2 + 7 = 9
    - É aqui que obtemos o valor final 9 em a0.

Resumo dos valores finais:

- s0 = 2
- s1 = 2
- a0 = 9

Cada instrução utilizou corretamente o formato R-type, I-type ou B-type do RISC-V, com o multiplexador de seleção de operandos e os sinais de controle apropriados no datapath para gerar os resultados esperados.

4. Conclusão

Os quatro novos módulos foram integrados corretamente ao pipeline, passando por todos os testes funcionais e de controle de fluxo, com o mantimento da organização e devidamente comentado. Esta implementação reforça a importância da modularização, do correto gerenciamento dos sinais de barramento e memória, e do tratamento preciso de desvios condicionais em um processador RISC-V em cinco estágios.